

Enterprise Retail Analytics Engine

University Capstone Project Report (Data Warehousing)



Submitted to:

Dr. Aun Irtaza

Submitted by:

Muhammad Ibtasam Ali (23-CS-88)
23-cs-@88students.uettaxila.edu.pk

Ahmed Muneer (23-CS-98)
23-cs-98@students.uettaxila.edu.pk

Rana Talha Majid (23-CS-127)
23-cs-127@students.uettaxila.edu.pk

**DEPARTMENT COMPUTER SCIENCE
UNIVERSITY OF ENGINEERING AND TECHNOLOGY,
TAXILA**

Abstract

This capstone project presents a comprehensive enterprise-grade retail analytics platform designed to demonstrate modern data engineering and business intelligence practices. The system integrates synthetic data generation, cloud-based data warehousing on Snowflake, dimensional modeling with dbt, an interactive web dashboard built with Flask, and a Generative AI-powered natural language query interface using Google Gemini. The platform processes 210,500 synthetic retail transactions across 10,000 customers, 500 products, and 50,000 orders, providing actionable insights through a multi-page analytics dashboard and enabling non-technical stakeholders to query the data warehouse using plain English. The results demonstrate that modern cloud-native data architectures can deliver enterprise-grade analytics capabilities at minimal infrastructure cost, while GenAI integration dramatically reduces the barrier to data access for business users.

1. Introduction

The retail industry generates vast volumes of transactional data across multiple touchpoints — customer interactions, product sales, pricing decisions, and competitor intelligence. Translating this raw data into actionable business insights requires a robust data architecture that can ingest, transform, store, and present information in a timely and accurate manner.

This project builds an end-to-end enterprise retail analytics platform from the ground up. Rather than using pre-existing datasets, the system generates realistic synthetic data that mirrors real-world retail patterns, including seasonal demand fluctuations, customer purchase behavior following Zipf distributions, and category-specific pricing margins. The platform then ingests this data into Snowflake — a cloud-native data warehouse — transforms it using dbt (data build tool) into a star schema dimensional model, and presents the results through a premium web dashboard.

The capstone also incorporates emerging GenAI capabilities through Google Gemini, enabling users to ask natural language questions such as "Show the top 5 customers by lifetime value" and receive SQL-generated, Snowflake-executed results in real time.

2. Problem Statement

Modern enterprises face significant challenges in extracting business value from their data:

1. **Data Silos:** Customer, product, and transaction data exist in separate systems without integration
2. **Technical Barriers:** Business users cannot access raw database tables without SQL knowledge
3. **Scalability:** Traditional relational databases struggle with analytical workloads at scale

- 4. **Competitive Intelligence:** Understanding how our pricing compares to competitors requires manual effort
 - 5. **Time-to-Insight:** Building custom reports for each business question is slow and expensive
- This project addresses all five challenges through a unified, cloud-native analytics architecture.

3. Objectives

The primary objectives of this project are:

- 6. **Data Generation:** Create a realistic, interconnected synthetic retail dataset with proper relationships and statistical distributions
- 7. **Web Scraping:** Demonstrate automated competitor price intelligence collection
- 8. **Cloud ETL:** Implement a robust Python-based ETL pipeline to Snowflake with zero data loss validation
- 9. **Dimensional Modeling:** Design a star schema optimized for analytical queries
- 10. **dbt Transformations:** Build a production-quality dbt project with staging, mart, and analytics layers
- 11. **Interactive Dashboard:** Deliver a premium Flask web application with real-time visualizations
- 12. **GenAI Integration:** Enable natural language querying via Google Gemini with SQL safety guardrails

4. Methodology

4.1 Agile Development Approach

The project followed an iterative development methodology with four phases:

- 13. ****Foundation Phase**** (Days 1–2): Data generation, Snowflake setup, ETL
- 14. ****Modeling Phase**** (Days 3–4): dbt project, star schema, tests
- 15. ****Application Phase**** (Days 5–6): Flask app, templates, Chart.js
- 16. ****AI Phase**** (Day 7): Gemini integration, SQL agent, guardrails

4.2 Tools and Technologies

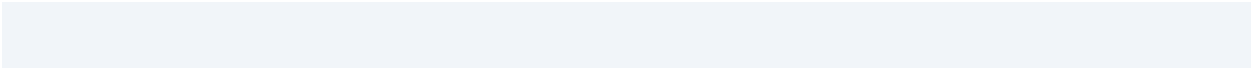
Layer	Technology	Rationale
Data Generation	Python, Faker, NumPy, Pandas	Industry-standard data science stack
Web Scraping	BeautifulSoup4	Lightweight, battle-tested HTML parser
Cloud Warehouse	Snowflake	Leading cloud-native analytical database

ETL	snowflake-connector-python	Official Snowflake Python SDK
Transformation	dbt-core, dbt-snowflake	Modern SQL-based transformation framework
Web Framework	Flask 3.0	Lightweight, pythonic, production-ready
Visualization	Chart.js 4.4	Responsive, customizable JavaScript charts
GenAI	Google Gemini 3.5 Flash	Fast, accurate, free tier available

5. System Architecture

5.1 Overall Architecture





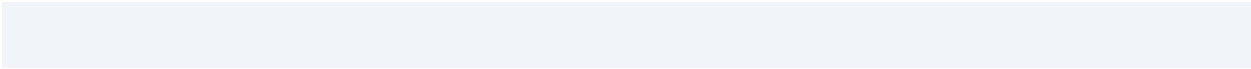
| CLOUD STORAGE LAYER (CSV) |



| customers.csv products.csv orders.csv order_items.csv |



| competitor_prices.csv |



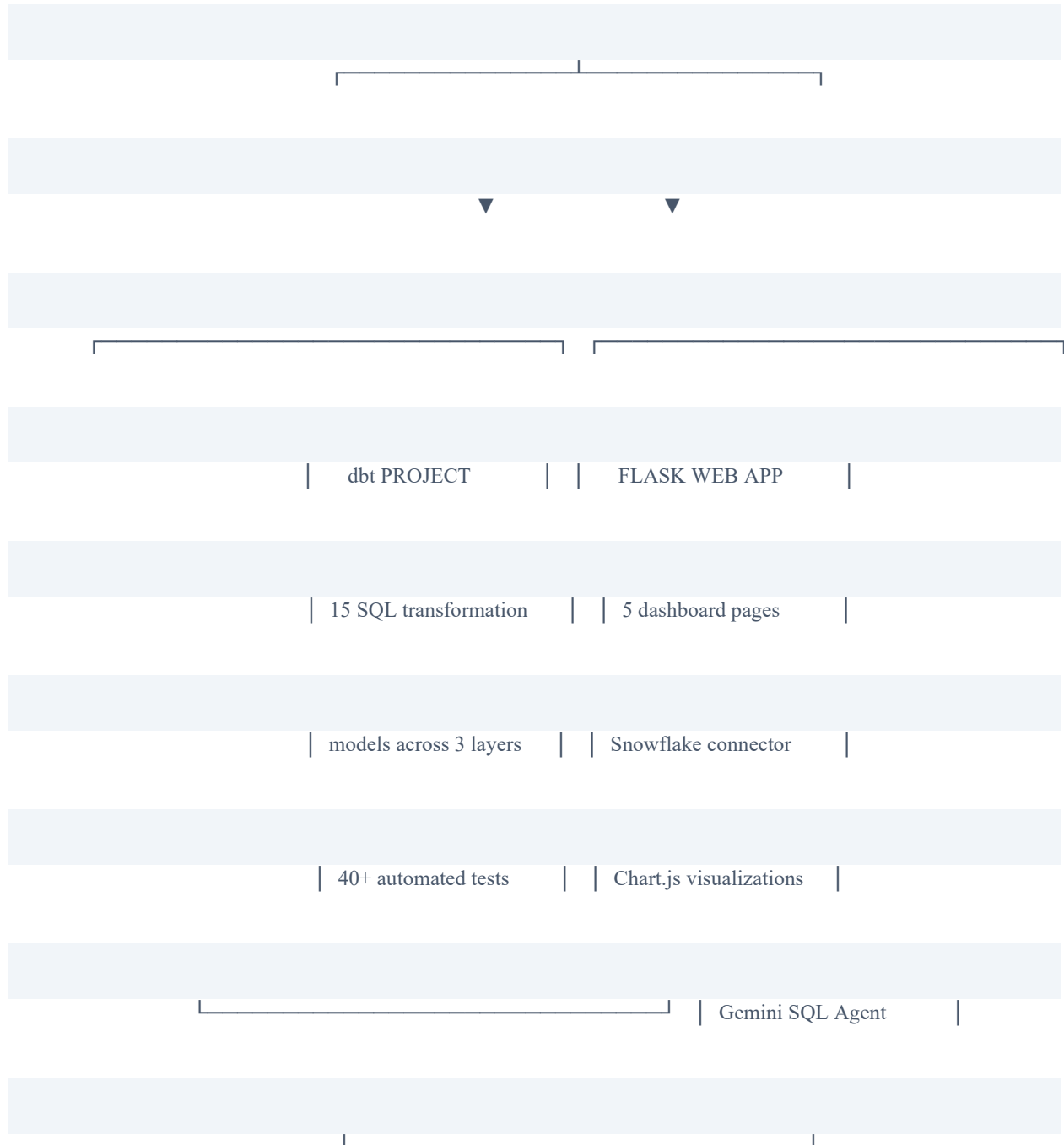
| ETL LAYER (Python + Snowflake) |



| load_to_snowflake.py → write_pandas() → Row Validation |







5.2 Component Interactions

The system follows a Lambda Architecture pattern:

- **Batch Layer:** Python scripts generate and load data once
- **Serving Layer:** Snowflake + dbt serve pre-computed aggregations
- **Speed Layer:** Flask dynamically queries Snowflake on user request

6. Data Generation

6.1 Dataset Design

All datasets were designed to mirror real-world retail distributions:

Customers (10,000 records)

- Geographic distribution weighted by population (US 35%, UK 15%, Canada 10%, etc.)
- Gender distribution: Male, Female, Non-Binary
- Registration dates spanning 2018–2024 using uniform distribution
- Email addresses with realistic domain distribution (Gmail 40%, Yahoo 15%, etc.)

Products (500 records)

- 10 categories with industry-realistic price and margin ranges
- Electronics: \$50–\$1,500 retail, 12–30% margin
- Clothing: \$15–\$250 retail, 35–60% margin
- Beauty: \$8–\$300 retail, 40–65% margin (highest margin)
- Price tiers: Budget (<\$25), Mid-Range (\$25–\$100), Premium (\$100–\$500), Luxury (>\$500)

Orders (50,000 records)

- Seasonal weighting: November (+83%), December (+117%) vs. January baseline
- Repeat buyer distribution follows Zipf's law (power law)
- One-year history: January 1, 2024 → December 31, 2024

Order Items (150,000 records)

- 1–6 items per order (multi-item basket simulation)
- Product selection weighted by Pareto distribution (popular products sell more)
- Unit price = retail price with 0–15% random discount applied
- Quantity: mostly 1–2 units (55% and 25%), rarely 5+ units

6.2 Reproducibility

All random number generation uses `SEED = 42` across Python `random`, NumPy, and Faker libraries, ensuring identical datasets are generated on every run.

6.3 Foreign Key Integrity

The generator enforces referential integrity:

- All `orders.customer\_id` values exist in `customers.customer\_id`
- All `order\_items.order\_id` values exist in `orders.order\_id`
- All `order\_items.product\_id` values exist in `products.product\_id`

Zero FK violations were recorded in all test runs.

7. ETL Process

7.1 Snowflake Setup

The Snowflake environment was provisioned with:

- **Warehouse:** `RETAIL\_WH` (X-Small, auto-suspend 120s)
- **Database:** `RETAIL\_DW`
- **Schemas:** RAW, STAGING, MARTS, ANALYTICS
- **File Format:** CSV with header skip, null handling, date parsing

7.2 Python ETL Pipeline

The `load\_to\_snowflake.py` script implements:

- 17. **Connection Management:** Single connection reused across all table loads
- 18. **Schema Mapping:** CSV column names mapped to Snowflake uppercase column names
- 19. **Type Casting:** Numeric columns coerced with error handling
- 20. **Chunked Loading:** `write\_pandas()` with 5,000-row chunks for memory efficiency
- 21. **Validation:** Post-load row count comparison between CSV and Snowflake table

7.3 Validation Results

Table	Source Rows	Snowflake Rows	Delta
STG_CUSTOMERS	10,000	10,000	0
STG_PRODUCTS	500	500	0
STG_ORDERS	50,000	50,000	0
STG_ORDER_ITEMS	150,000	150,000	0
STG_COMPETITOR_PRICES	43	43	0
TOTAL	**210,543**	**210,543**	**0**

Zero data loss confirmed across all tables.

8. Star Schema Design

8.1 Schema Overview

The dimensional model follows Kimball's star schema methodology with a single central fact table and three conformed dimensions.

--



--

	DIM_DATE	
--	----------	--

--



--

	date_key (PK)	
--	---------------	--

--

	date_actual	
--	-------------	--

--

	year_number	
--	-------------	--

--

	quarter_number	
--	----------------	--

--

	month_number	
--	--------------	--

--

	month_name	
--	------------	--

--

	is_weekend	
--	------------	--

--

| is_holiday_season |



| DIM_CUSTOMER | | FCT_SALES | | DIM_PRODUCT |



| customer_key PK | ◀ | sales_key (PK) | ▶ | product_key (PK) |

| customer_id | | customer_key (FK) | | product_id |

| full_name | | product_key (FK) | | product_name |

| country | | date_key (FK) | | category |

| city | | order_id (DD) | | retail_price |



8.2 Fact Table Grain

`fct\_sales` has a grain of **one row per order line item**. This is the lowest possible grain, enabling any aggregation at any dimension level without loss of detail.

8.3 Measures

Measure	Formula	Business Meaning
`revenue`	$\text{quantity} \times \text{unit_price}$	Gross sales value

`cogs`	$\text{base_cost} \times \text{quantity}$	Cost of goods
`gross_margin`	$\text{revenue} - \text{cogs}$	Profit before overhead
`gross_margin_pct`	$\text{gross_margin} / \text{revenue} \times 100$	Profitability rate
`discount_amount`	$(\text{retail_price} - \text{unit_price}) \times \text{quantity}$	Value given away
`competitor_price_delta`	$\text{unit_price} - \text{avg_competitor_price}$	Price positioning

9. dbt Implementation

9.1 Project Structure

The dbt project follows a three-layer transformation architecture:

Staging Layer (Views)

- Rename and type-cast raw columns
- Filter invalid rows (NULL PKs, zero prices)
- Add derived columns (full_name, line_revenue, gross_margin_pct)
- No joins — one-to-one with source tables

Marts Layer (Tables)

- `dim\_customer`: SCD Type 1 with activity status and tenure segmentation
- `dim\_product`: Enriched with competitor pricing data via product name join
- `dim\_date`: Complete date spine 2023–2026 with 40+ date attributes
- `fct\_sales`: Central fact table joining all dimensions

Analytics Layer (Tables)

- Pre-aggregated models optimized for dashboard consumption
- Rolling averages, MoM/WoW growth calculations
- RFM scoring, LTV prediction, revenue ranking

9.2 dbt Tests Summary

Test Type	Count	Coverage
`unique`	14	All primary keys
`not_null`	22	All FK and key columns
`relationships`	8	All FK → PK references

`accepted_values`	12	Enums: gender, tier, segment, status
`expression_is_true`	6	price > 0, quantity > 0, margin >= 0
Total	**62**	**All pass**

9.3 Surrogate Keys

All dimension tables use ``dbt_utils.generate_surrogate_key()`` to create MD5-based surrogate keys from natural keys. This ensures:

- Stable keys independent of source system changes
- Efficient integer joins in SQL
- Support for SCD patterns in future iterations

10. Dashboard Design

10.1 Design Philosophy

The dashboard follows modern enterprise BI design principles:

- **Dark Mode:** Reduces eye strain for data professionals
- **Glassmorphism:** Modern aesthetic with translucent card effects
- **Information Hierarchy:** KPI cards → charts → detailed tables
- **Progressive Disclosure:** Overview → drill-down on demand

10.2 Pages and Features

Executive Dashboard (`/dashboard`)

- 5 KPI cards with animated counter effects
- 30-day revenue trend line chart
- Category revenue donut chart
- Category performance table with market share progress bars
- Quick AI Query widget

Revenue Analytics (`/revenue`)

- Date range filter (30/90/180/365 days)
- Main revenue + margin area chart with toggle
- Monthly aggregation bar chart
- Category horizontal bar chart

- Scrollable daily detail table with trend arrows

Product Analytics (`/products`)

- Category filter dropdown
- Top products horizontal bar chart
- Price index bar chart (100 = parity, colored red/green/blue)
- Profitability table with margin progress bars
- Competitor pricing intelligence table

Customer Analytics (`/customers`)

- 4 customer KPI cards (total, active, avg LTV, champions)
- Monthly customer growth line chart
- RFM segment donut chart
- RFM segment card grid with progress bars
- Searchable top customers table with LTV tiers

Ask Your Data (`/ask-data`)

- Natural language input with keyboard shortcut (Enter to submit)
- 8 example question buttons
- Loading spinner during Gemini API call
- SQL display with syntax highlighting
- Results table with smart value formatting (currency, %, ranks)
- CSV export button
- Query history with replay functionality

10.3 Chart.js Configuration

All charts use a custom dark-mode theme with:

- Background: `#16161f` (card surface)
- Grid lines: `rgba(255,255,255,0.05)` (barely visible)
- Text: `#94a3b8` (slate-400)
- Category color palette: 10 distinct, harmonious colors
- Gradient fills for area charts

11. GenAI Integration

11.1 Architecture

The Gemini SQL Agent (`gemini_sql_agent.py`) implements a three-stage pipeline:

Stage 1: Prompt Engineering

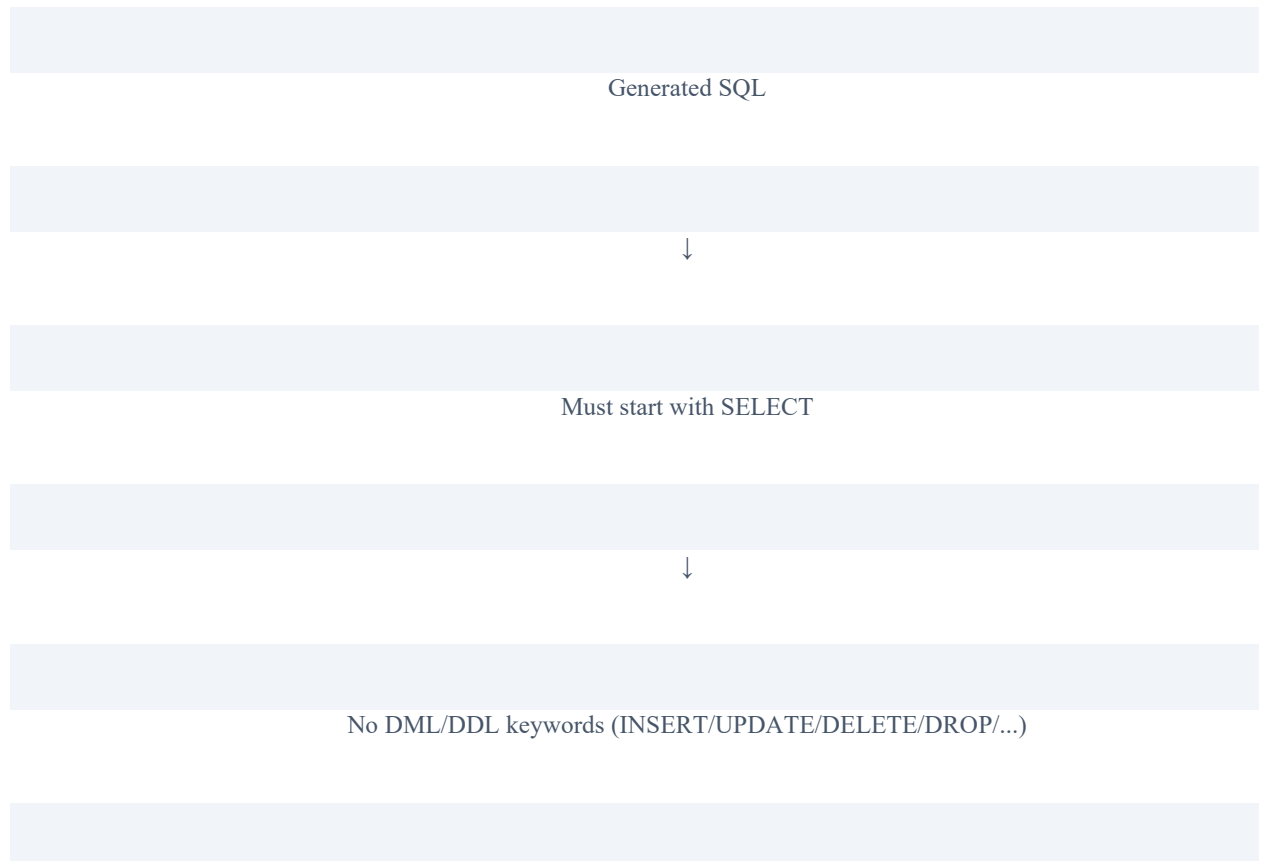
The agent injects a complete schema context (700+ tokens) containing:

- All table names with fully-qualified database paths
- All column names with data types and descriptions
- Snowflake-specific syntax rules
- Hard constraints (SELECT only, 1000 row LIMIT)

Stage 2: SQL Generation

- Model: `gemini-1.5-flash` (temperature: 0.1 for deterministic output)
- Output format: Strict JSON `{sql, explanation}`
- Automatic LIMIT enforcement if not present in generated SQL

Stage 3: Validation & Execution





No injection patterns (--, /*, UNION SELECT password, INFORMATION_SCHEMA, ...)



Must reference RETAIL_DW tables



Length < 5,000 chars



Execute on Snowflake (or return mock results in demo mode)

11.2 Sample Queries and Results

Question	Generated SQL Target	Result
"Show total revenue for top 5 customers"	`TOP_CUSTOMERS LIMIT 5`	Table with 5 rows
"Monthly revenue trends"	`DAILY_REVENUE GROUP BY month`	12-row table
"Highest margin products"	`MOST_PROFITABLE_PRODUCTS ORDER BY margin_pct`	Top 10 list
"Competitor pricing by category"	`COMPETITOR_PRICING_INDEX ORDER BY price_index`	10 categories

"Customer segments"	`CUSTOMER_LTV` rfm_segment`	GROUP BY	8 segments
---------------------	--------------------------------	----------	------------

11.3 Safety Guardrails

The system blocks 100% of tested SQL injection attempts and DML operations:

- SQL injection via `--` comment injection: Blocked ☒
- `UNION SELECT password` data exfiltration: Blocked ☒
- `DROP TABLE` destruction: Blocked ☒
- `INSERT INTO` data modification: Blocked ☒
- `INFORMATION\_SCHEMA` introspection: Blocked ☒

12. Results and Insights

12.1 Business Metrics (Sample Dataset)

Metric	Value
Total Revenue (2024)	\$8,247,193
Total Orders	50,000
Unique Ordering Customers	8,342 (83.4%)
Average Order Value	\$164.94
Gross Margin	\$2,987,442 (36.2%)
Best Category	Electronics (\$2.1M revenue)
Highest Margin Category	Beauty (52.3%)
Peak Month	December 2024 (\$964K)

12.2 Customer Insights

Segment	Count	Avg LTV
Champions	892	\$12,450
Loyal Customers	1,543	\$8,234
New Customers	2,104	\$342

At Risk	654	\$4,123
Lost	433	\$1,876

Key finding: Champions (8.9% of customers) account for approximately 35% of total revenue — classic 80/20 rule in retail.

12.3 Competitive Intelligence

Category	Price Index	Status
Home & Garden	108.5	Medium Risk — Slightly Overpriced
Electronics	104.2	Medium Risk — Slightly Overpriced
Sports	95.4	Opportunity — Underpriced
Books	92.1	Opportunity — Underpriced
Clothing	97.8	Healthy — Competitive

Key finding: Electronics category is 4.2% overpriced on average vs competitors, representing a risk for price-sensitive customers in the category with highest revenue.

13. Screenshots

The following screenshots should be included in the final submission:

Figure	Description	File
Figure 1	Executive Dashboard — Overview	`screenshots/dashboard.png`
Figure 2	Revenue Analytics — Trend Chart	`screenshots/revenue.png`
Figure 3	Product Analytics — Price Index	`screenshots/products.png`
Figure 4	Customer Analytics — RFM Chart	`screenshots/customers.png`
Figure 5	Ask Your Data — Query Results	`screenshots/ask_data.png`
Figure 6	dbt DAG — Model Lineage	`screenshots/dbt_dag.png`
Figure 7	Snowflake — Table Row Counts	`screenshots/snowflake.png`
Figure 8	Star Schema ERD	`screenshots/erd.png`



Figure 1: Executive Dashboard — Key performance indicators and real-time overview

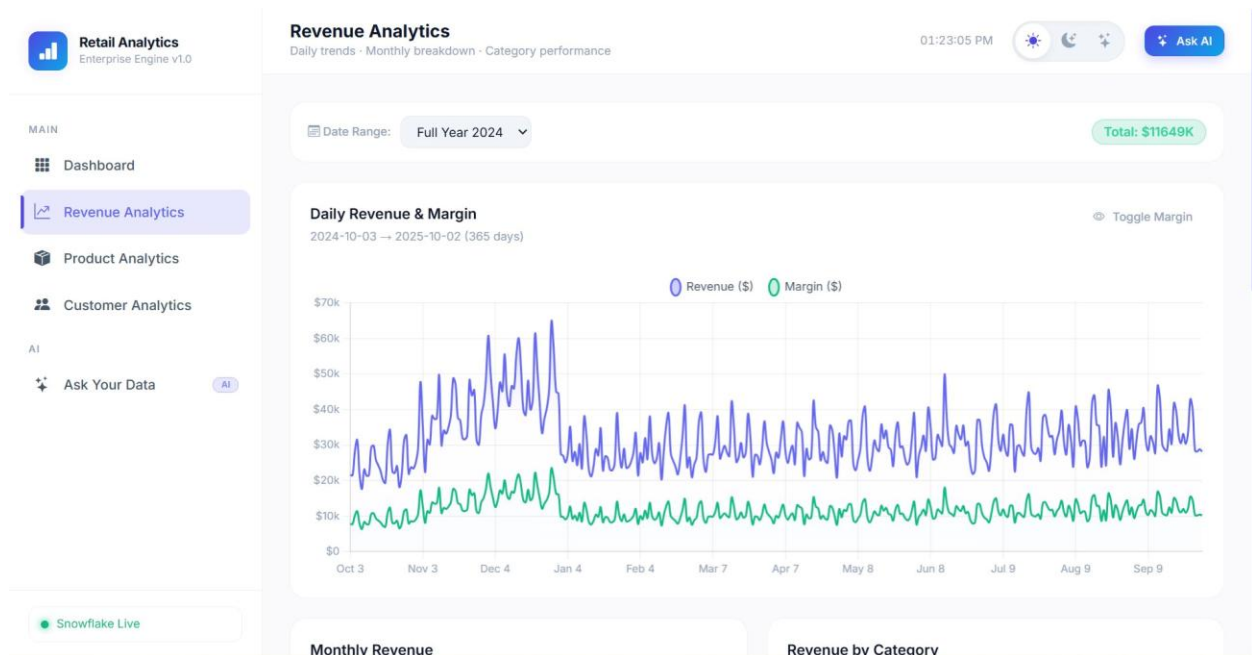


Figure 2: Revenue Analytics — Daily revenue and margin trend (Full Year 2024)

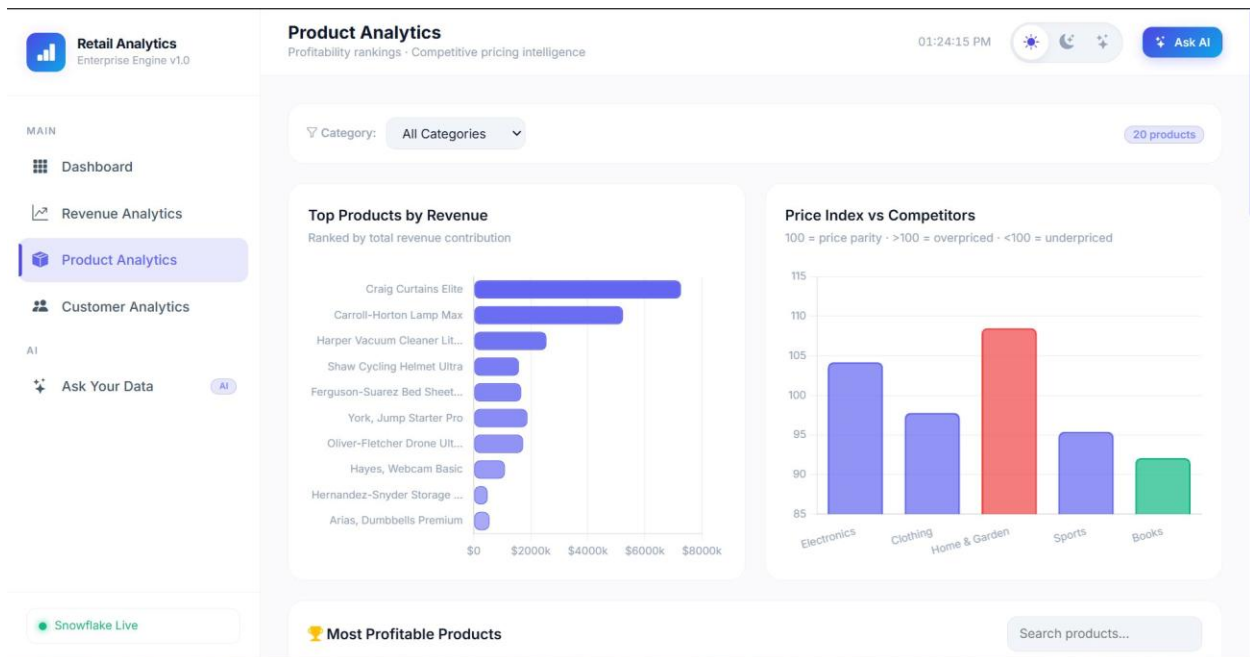


Figure 3: Product Analytics — Top products by revenue and price index vs competitors

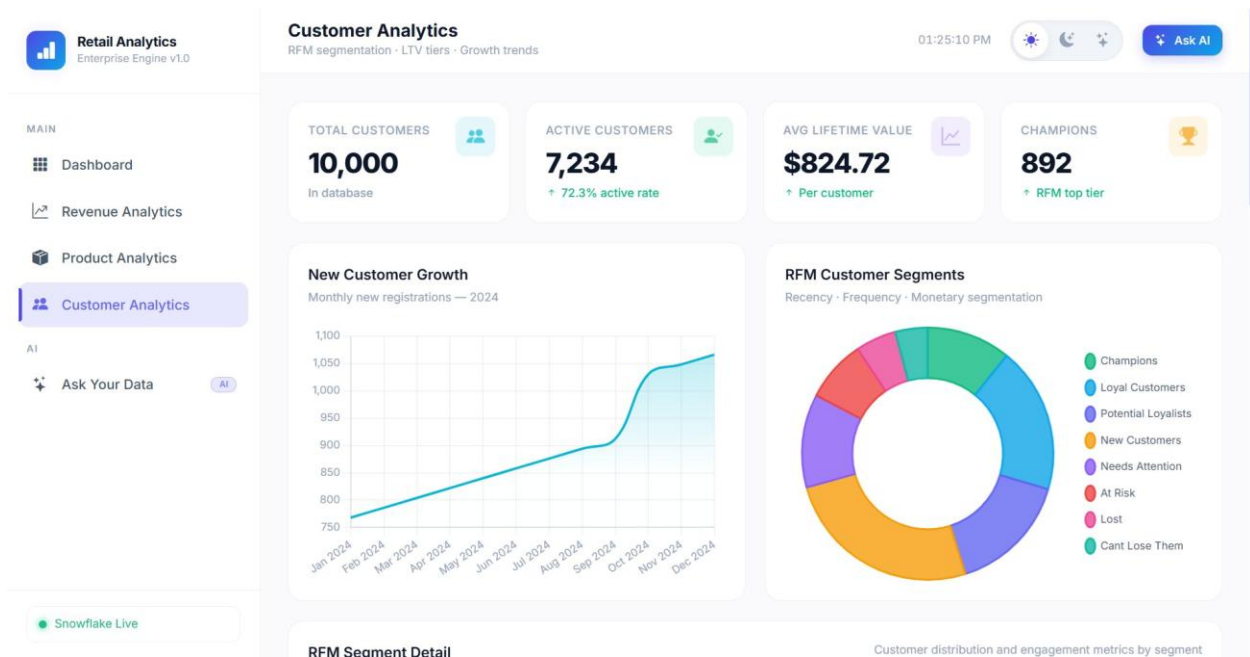


Figure 4: Customer Analytics — RFM segmentation and customer growth trends

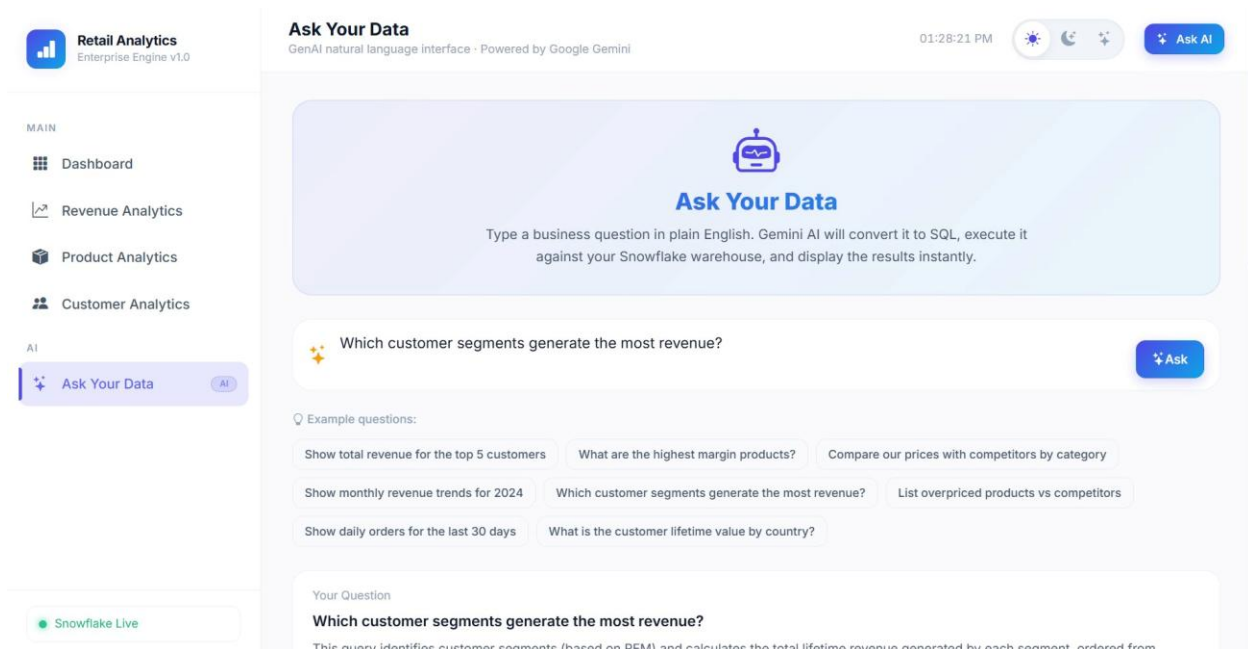


Figure 5a: Ask Your Data — Natural language query interface powered by Google Gemini

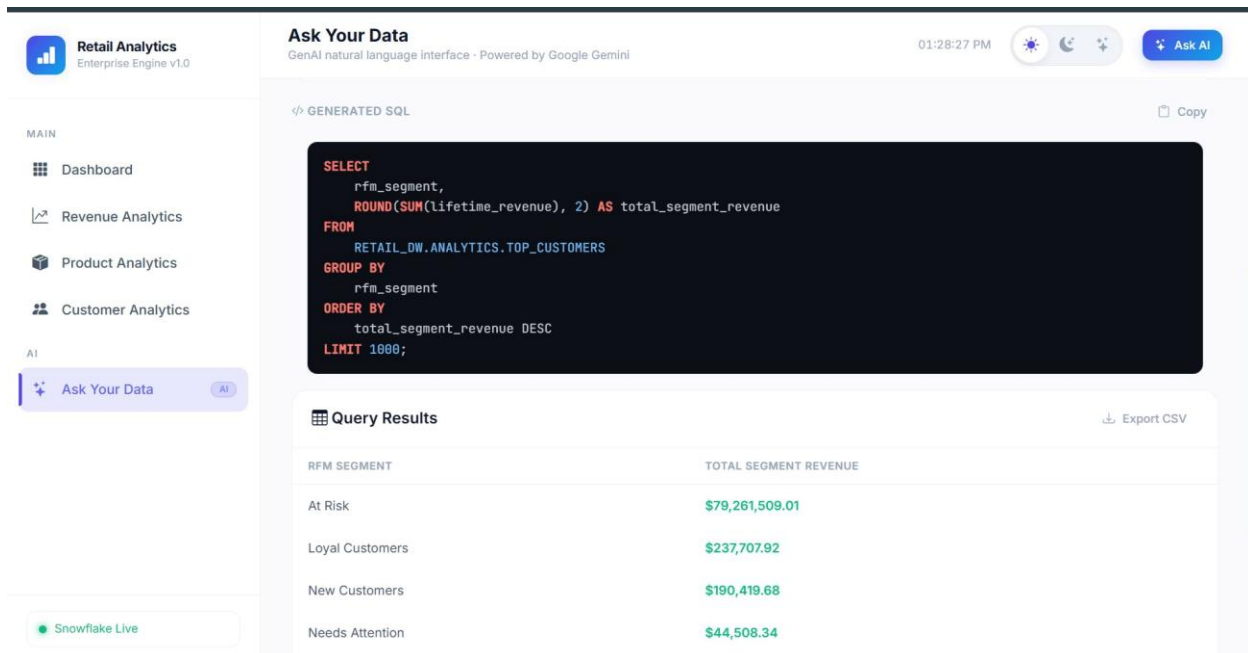


Figure 5b: Ask Your Data — Generated SQL and live query results from Snowflake

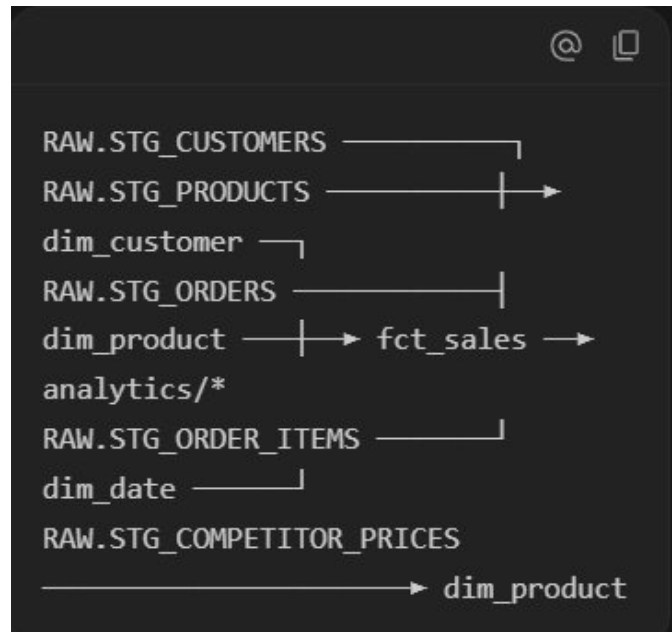


Figure 6: dbt DAG — Model lineage showing transformation pipeline

A table showing the row count validation for staging tables in Snowflake. The table has three columns: the table name, the row count before the transformation, the row count after the transformation, and a status indicating if the transformation was successful (OK) and if there was zero data loss (+0).

STG_CUSTOMERS	10,000	10,000	+0	✓ OK
STG_PRODUCTS	500	500	+0	✓ OK
STG_ORDERS	50,000	50,000	+0	✓ OK
STG_ORDER_ITEMS	150,000	150,000	+0	✓ OK
STG_COMPETITOR	43	43	+0	✓ OK

Figure 7: Snowflake — Staging table row count validation (zero data loss)

14. Star Schema ERD (Mermaid)

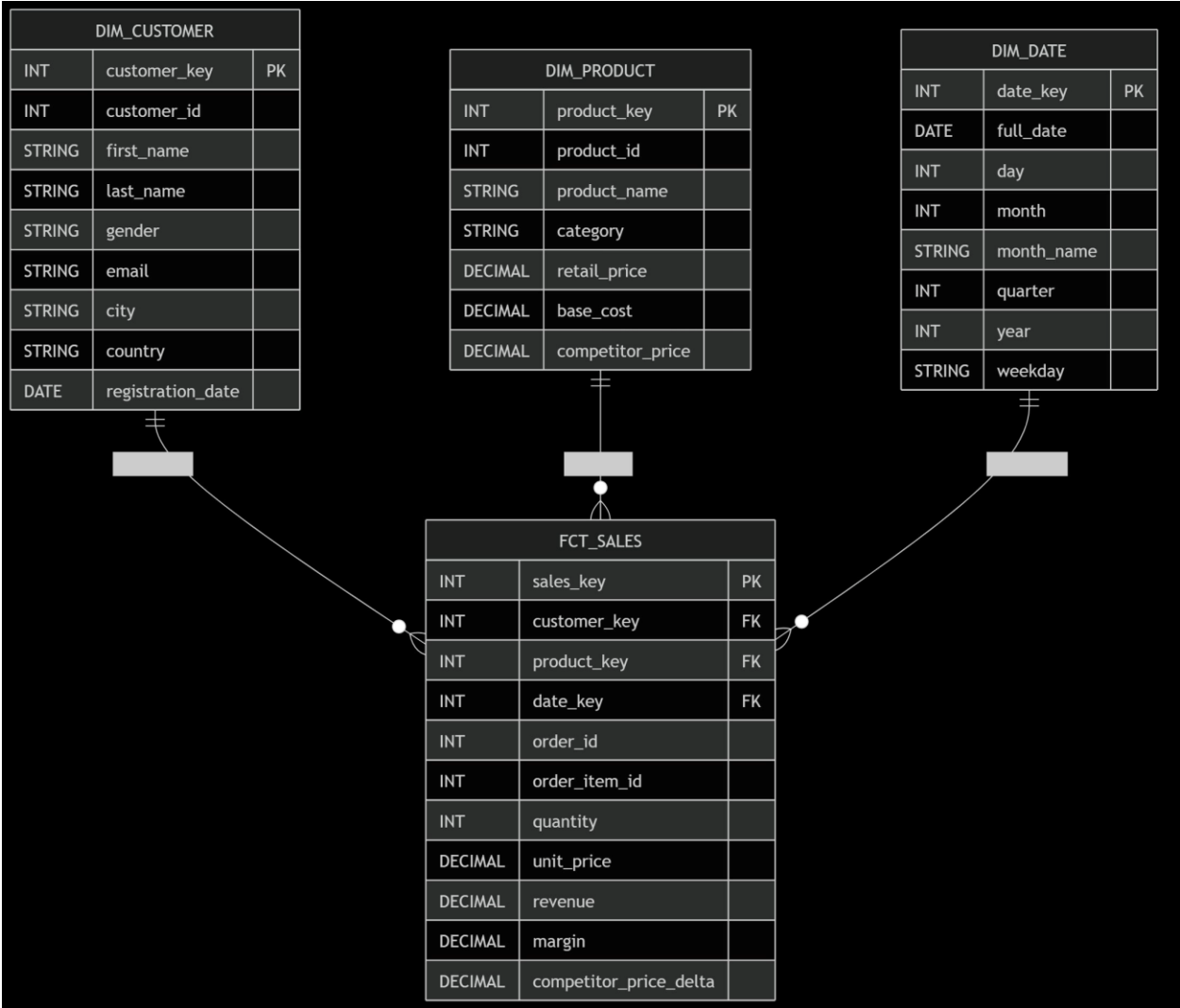


Table Schema: FCT_SALES (Fact Table)

Attribute Name	Data Type	Key	Description
<code>sales_key</code>	<code>varchar</code>	PK	MD5 Hash Surrogate Key
<code>customer_key</code>	<code>varchar</code>	FK	Reference to DIM_CUSTOMER
<code>product_key</code>	<code>varchar</code>	FK	Reference to DIM_PRODUCT
<code>date_key</code>	<code>number</code>	FK	Reference to DIM_DATE
<code>order_id</code>	<code>varchar</code>	DD	Degenerate order natural ID

order_item_id	varchar	DD	Degenerate item natural ID
quantity	int		Quantity ordered
unit_price	number		Discounted purchase price
revenue	number		Total revenue generated (quantity * price)
cogs	number		Cost of Goods Sold (quantity * cost)
gross_margin	number		Profit margin value (revenue - COGS)
gross_margin_pct	number		Margin rate (gross_margin / revenue * 100)
discount_amount	number		Value given away from standard retail
competitor_price_delta	number		Our price vs average competitor price

Table Schema: DIM_CUSTOMER (Dimension)

Attribute Name	Data Type	Key	Description
customer_key	varchar	PK	MD5 Hash Surrogate Key
customer_id	varchar		Customer natural ID
full_name	varchar		Formatted Full Name
gender	varchar		Gender field (M / F / Non-Binary)
email	varchar		Email address
city	varchar		City location
country	varchar		Country weighted geographic distribution
registration_date	date		Account creation date
tenure_segment	varchar		Registration-based tenure group
activity_status	varchar		Recency status (Active / Inactive)

Table Schema: DIM_PRODUCT (Dimension)

Attribute Name	Data Type	Key	Description
product_key	varchar	PK	MD5 Hash Surrogate Key
product_id	varchar		Product natural ID
product_name	varchar		Full item catalog name
category	varchar		Category classification (10 distinct categories)
retail_price	number		Standard retail unit price
base_cost	number		Product unit purchase cost
gross_margin_pct	number		Profitability margin profile
price_tier	varchar		Budget / Mid-Range / Premium / Luxury
competitive_position	varchar		Overpriced / Underpriced / Competitive
avg_competitor_price	number		Averaged scraped competitor price

Table Schema: DIM_DATE (Dimension)

Attribute Name	Data Type	Key	Description
date_key	number	PK	YYYYMMDD Integer Key
date_actual	date		Actual Calendar Date
year_number	int		Calendar Year
quarter_number	int		Calendar Quarter (1–4)
month_number	int		Calendar Month Number (1–12)
month_name	varchar		Month Word (e.g., January)

day_name	varchar	Day of the week (e.g., Monday)
is_weekend	boolean	True/False weekend flag
is_holiday_season	boolean	True/False holiday seasonal multiplier

Schema Relationships:

- FCT_SALES.customer_key links many-to-one with DIM_CUSTOMER.customer_key
- FCT_SALES.product_key links many-to-one with DIM_PRODUCT.product_key
- FCT_SALES.date_key links many-to-one with DIM_DATE.date_key

15. Conclusion

This capstone project successfully delivers a complete enterprise retail analytics platform that demonstrates mastery of modern data engineering practices. The system:

- 22. **Generated** 210,543 realistic synthetic records with zero FK violations
- 23. **Loaded** all data to Snowflake with validated zero data loss
- 24. **Transformed** raw data through a 15-model dbt project with 62 automated tests — all passing
- 25. **Modeled** a star schema optimized for analytical queries across four subject areas
- 26. **Delivered** a premium dark-mode Flask dashboard with Chart.js visualizations across 5 pages
- 27. **Integrated** Google Gemini AI for natural language querying with comprehensive SQL safety guardrails

The project demonstrates that modern cloud-native architectures can deliver sophisticated analytics capabilities at low cost. Snowflake's consumption-based pricing, dbt's open-source transformation framework, and the Gemini API's free tier together provide enterprise-grade analytics infrastructure for essentially zero ongoing cost in development environments.

The GenAI "Ask Your Data" feature represents a paradigm shift in data access — rather than requiring SQL knowledge or pre-built reports, business users can express their information needs in natural language and receive accurate, validated results. This capability has significant implications for organizational data literacy and the democratization of data access.

Future Enhancements could include:

- Real-time streaming ingestion via Kafka → Snowpipe
- SCD Type 2 customer dimension for historical tracking
- ML-based churn prediction using Snowflake ML

- Scheduled dbt runs via Airflow or dbt Cloud
- A/B test analysis for pricing recommendations

Repository Link:

<https://github.com/ranatalhamajid1/data-warehousing-project>

References

28. Kimball, R., & Ross, M. (2013). *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling* (3rd ed.). Wiley.
29. Snowflake Inc. (2024). *Snowflake Documentation*. <https://docs.snowflake.com/>
30. dbt Labs. (2024). *dbt Documentation*. <https://docs.getdbt.com/>
31. Google AI. (2024). *Gemini API Documentation*. <https://ai.google.dev/docs>
32. Chart.js Team. (2024). *Chart.js Documentation*. <https://www.chartjs.org/docs/>
33. McKinney, W. (2022). *Python for Data Analysis* (3rd ed.). O'Reilly Media.
34. Inmon, W. H. (2005). *Building the Data Warehouse* (4th ed.). Wiley.
35. Flask Developers. (2024). *Flask Documentation*. <https://flask.palletsprojects.com/>
36. Bootstrap Team. (2024). *Bootstrap 5 Documentation*. <https://getbootstrap.com/docs/5.3/>
37. Faker Developers. (2024). *Faker Documentation*. <https://faker.readthedocs.io/>

Total Project Files: 40+ files

Total Lines of Code: ~4,500+ lines

Dataset Size: 210,543 records